SUPERVISED LEARNING — II

K-Nearest Neighbors

The classifier that never builds a model — it just remembers

Lazy Learning

Distance-Based

Majority Vote

Why the kNN algorithm is called a lazy learner?

- The eager learners follow the general steps of machine learning,
- i.e. perform an abstraction of the information obtained from the input data and then follow it through by a generalization step.
- In the kNN algorithm, these steps are completely skipped.
- It stores the training data and directly applies the philosophy of nearest neighbourhood finding to arrive at the classification.
- there is no learning happening in the real sense.
- Therefore, kNN falls under the category of lazy learner.

WHY KNN?

A different way of learning: lazy vs. eager

EAGER LEARNERS

ID3 · CART · Naïve Bayes

- Build an explicit model during training — a tree of rules, or a set of probabilities.
- Training takes time; prediction is fast — just apply the learned rule.
- The model can be inspected, pruned, and reused without the original data.

LAZY LEARNER

K-Nearest Neighbors

- Builds no model at all — it just stores the training data.
- "Training" is instant; all the work happens at prediction time.
- To classify a new point, it looks at which labeled points sit closest to it.

Guiding question: if you can't build a formula or a tree, how would you guess a new point's label using only the data you already have?

k-Nearest Neighbour (kNN) Algorithm

- The kNN algorithm is a simple but extremely powerful supervised learning algorithm.
- K-NN algorithm
  - stores all the available data and
  - classifies a new data point based on the similarity.
- The kNN is used in classifications or predictions, the grouping of an individual data point.
- This means when new data appears then it can be easily classified into a well suite Category/Class.

kNN algorithm

- Input: Training data set, test data set (or data points), value of ‘k’ (i.e. number of nearest neighbours to be considered)
- Steps:
- Do for all test data points
- Calculate the distance (usually Euclidean distance) of the test data point from the different training data points.
- Find the closest ‘k’ training data points, i.e. training data points whose distances are least from the test data point.
- If k = 1
- Then assign class label of the training data point to the test data point
- Else
- Whichever class label is mostly present in the training data points, assign that class label to the test data point
- End do

kNN Algorithm – Selecting k value

- If the value of k is very large (in the extreme case equal to the total number of records in the training data), the class label of the majority class of the training data set will be assigned to the test data regardless of the class labels of the neighbours nearest to the test data.
- If the value of k is very small (in the extreme case equal to 1), the class value of a noisy data or outlier in the training data set which is the nearest neighbour to the test data will be assigned to the test data.
- The best k value is somewhere between these two extremes.

kNN Algorithm – Selecting k value

- Few strategies are adopted by machine learning practitioners to arrive at a value for k.
- 1. k equal to the square root of the number of training records.
- 2. test several k values on a variety of test data sets and choose the one that delivers the best performance.
- 3. choose a larger value of k, but apply a weighted voting process in which the vote of close neighbours is considered more influential than the vote of distant neighbours.

Choosing K is a bias–variance tradeoff

e.g. K = 1

Small K

Flexible, jagged boundary. Low bias, high variance — very sensitive to noise. Easy to overfit.

e.g. K = n/2

Large K

Smooth boundary. High bias, low variance — may blur real local structure. Easy to underfit.

tuned K

Just right

Found via cross-validation, not a formula

Practical rules: prefer an odd K to avoid tie votes • a common starting point is K ≈ √n (here, n=8 → K≈3) • sweep K and pick the value with lowest cross-validation error.

Classify a point by asking its neighbors

To classify a new point x, find the K training points closest to x, and assign the class that most of them belong to.

1

Store

Keep the entire labeled training set in memory. No training phase, no parameters learned.

2

Measure

For a new point, compute its distance to every stored point (usually Euclidean).

3

Vote

Take the K closest points and assign the majority class among them.

Distance metrics: how “close” is defined

Euclidean

$$d(x,y) = \sqrt{\Sigma(x_i - y_i)^2}$$

Straight-line distance. The default choice for continuous, similarly-scaled features.

Manhattan

$$d(x,y) = \Sigma|x_i - y_i|$$

Sum of absolute differences — like navigating a city grid. Less sensitive to outliers.

Minkowski

$$d(x,y) = (\Sigma|x_i - y_i|^p)^{(1/p)}$$

A generalization: p=2 gives Euclidean, p=1 gives Manhattan.

Example

Given Query: x=(Maths=6, CS=8) (X2, Y2)

Step 1: Select K neighbors. K=3

Sr. No.	Maths	CS	Result
1	4	3	Fail
2	6	7	Pass
3	7	8	Pass
4	5	5	Fail
5	8	8	Pass

Euclidean Distance Formula:

$$d = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$$

- X_2, y_2 = Observe Valúes
- X1. y1 = Actual Valúes
- d = distance between (x11, v11) and (x22, v22)

Step 2: Calculate the Euclidean distance of **K number of neighbors.**

1. $\sqrt{[(6-4)^2 + (8-3)^2]} = \sqrt{29} = 5.38$
2. $\sqrt{[(6-6)^2 + (8-7)^2]} = 1$
3. $\sqrt{[(6-7)^2 + (8-8)^2]} = 1$
4. $\sqrt{[(6-5)^2 + (8-5)^2]} = \sqrt{10} = 3.16$
5. $\sqrt{[(6-8)^2 + (8-8)^2]} = 2$

Step 3: As per Result, **2. Pass 3. Pass 5. Pass**

We have, **3 Pass & 0 Fail** So, **3>0**

Probability of Pass is more.

Hence, In Given Query x=(Maths=6, CS=8) is Pass.

WORKED EXAMPLE

Will this student pass?

Two features: hours studied per week (X) and assignments completed out of 10 (Y). The new student studied 5 hours and completed 5 assignments — Pass or Fail?

Student	X (hrs)	Y (assign.)	Result
S1	6	6	Pass
S2	3	5	Pass
S3	5	2	Fail
S4	8	3	Fail
S5	1	4	Fail
S6	6	0	Fail
S7	10	8	Fail
S8	11	3	Pass
New?	**5**	**5**	**?**



WORKED EXAMPLE — STEP 1

Compute the Euclidean distance to every point

$$d(New, S_i) = \sqrt{(5 - x_i)^2 + (5 - y_i)^2}$$

Student	X	Y	Result	Calculation	Distance
S1	6	6	Pass	$\sqrt{(1^2+1^2)}$	**1.41**
S2	3	5	Pass	$\sqrt{(2^2+0^2)}$	**2.00**
S3	5	2	Fail	$\sqrt{(0^2+3^2)}$	**3.00**
S4	8	3	Fail	$\sqrt{(3^2+2^2)}$	**3.61**
S5	1	4	Fail	$\sqrt{(4^2+1^2)}$	**4.12**
S6	6	0	Fail	$\sqrt{(1^2+5^2)}$	**5.10**
S7	10	8	Fail	$\sqrt{(5^2+3^2)}$	**5.83**
S8	11	3	Pass	$\sqrt{(6^2+2^2)}$	**6.32**

WORKED EXAMPLE — STEP 2

Sort by distance, then vote (K = 3)

Rank	Student	Distance	Result	In K=3?
1	**S1**	1.41	Pass	✓
2	**S2**	2.00	Pass	✓
3	**S3**	3.00	Fail	✓
4	S4	3.61	Fail	
5	S5	4.12	Fail	
6	S6	5.10	Fail	
7	S7	5.83	Fail	
8	S8	6.32	Pass	

Nearest 3 neighbors

S1 (Pass) · S2 (Pass) · S3 (Fail)

Vote: 2 Pass vs 1 Fail

→ Majority class wins

Prediction: PASS

THE ROLE OF K

The same point, two different answers

As K grows, the neighborhood expands — and the majority vote can flip.

K = 3

S1 (Pass) · S2 (Pass) · S3 (Fail)

Vote: 2 Pass vs 1 Fail

Prediction: PASS

K = 7

S1 (Pass) · S2 (Pass) · S3 (Fail) · S4 (Fail) · S5 (Fail) · S6 (Fail) · S7 (Fail)

Vote: 2 Pass vs 5 Fail

Prediction: FAIL

BEFORE YOU USE KNN

Three things that quietly break KNN

1	**Feature scaling**	Distance is dominated by whichever feature has the largest numeric range. Always standardize or normalize features first.
2	**Curse of dimensionality**	In high dimensions, distances stop being meaningful — all points start looking roughly equidistant.
3	**Computational cost**	No training cost, but every prediction compares against all N stored points. KD-trees / Ball-trees speed this up in practice.

Strengths and weakness of the kNN algorithm

- Strengths
- Extremely simple algorithm – easy to understand
- Very effective in certain situations,
- Very fast or almost no time required for the training phase
- Weaknesses
- Does not learn anything in the real sense.
- Classification is done completely on the basis of the training data.
- If the training data does not represent the problem domain systematically, the algorithm fails to make an effective classification.
- Also, a large amount of computational space is required to load the training data for classification.

Application of the kNN algorithm

- The recommender systems, recommend users, with different items which are similar to a particular item that the user seems to like.
- The liking pattern may be revealed from past purchases or browsing history and the similar items are identified using the kNN algorithm.
- Information retrieval (concept search), Searching documents/ contents similar to a given document/content.

REGRESSION, EXPLAINED

Linear Regression vs Logistic Regression

Predicting numbers vs. predicting categories — where each model fits, why one breaks down for classification, and how to build both in Python.

Class Session • Predictive Modeling Basics

TODAY'S ROADMAP

What We'll Cover

1. **Linear Regression**
   What it predicts, and a small worked dataset
2. **Where It Breaks**
   Why a straight line fails for yes/no outcomes
3. **Logistic Regression**
   The sigmoid fix, and its own worked dataset
4. **Python, Side by Side**
   Fitting both models on tiny datasets
5. **When to Use Which**
   Real use-cases + head-to-head comparison

What Is Linear Regression?

Predicts a NUMBER

Output can be any continuous value (price, score, temperature).

Fits a straight line

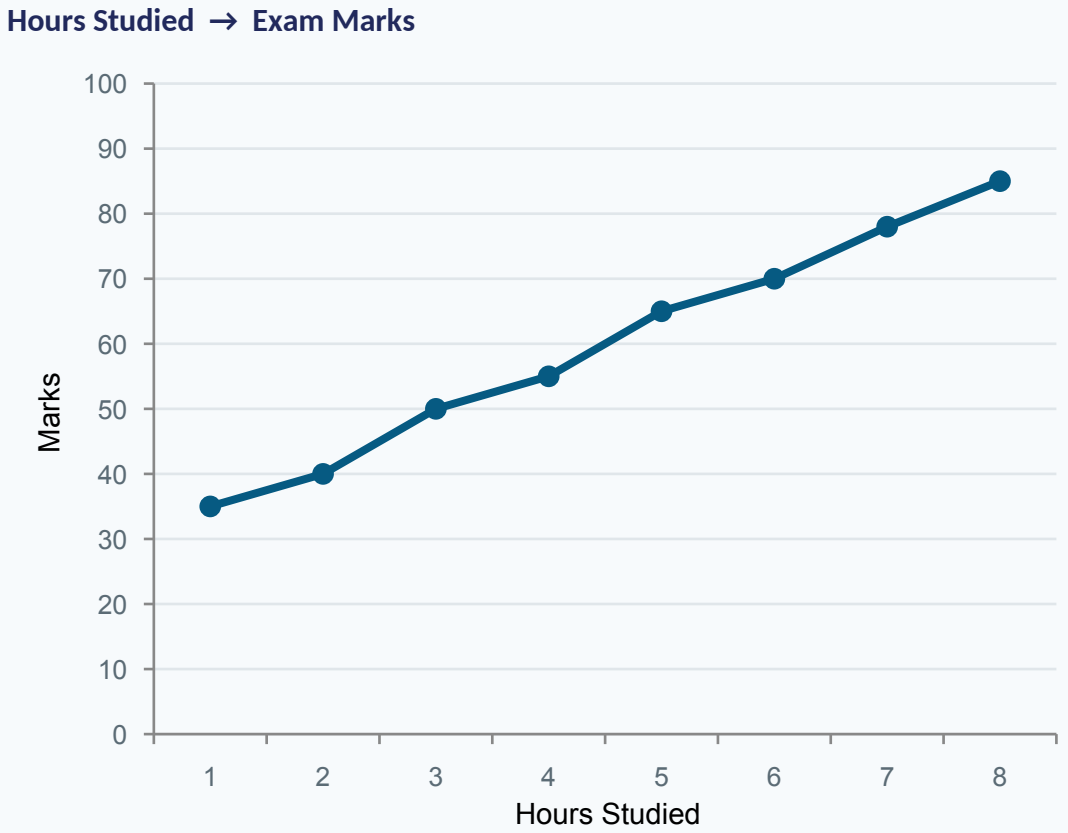
Finds the best-fitting line through the data points.

Answers "how much?"

e.g. How many marks? What salary? What price?

Equation

$$y = b_0 + b_1 x$$



PART 1 · WORKED EXAMPLE

A Tiny Dataset: Study Hours → Marks

Hours Studied	Marks Obtained
1	35
2	40
3	50
4	55
5	65
6	70
7	78
8	85

What the model learns

- The line that best fits these 8 points, minimizing squared error.
- Roughly: Marks ≈ 27 + 7.3 × Hours
- Predict for a NEW student: 4.5 hours → Marks ≈ 27 + 7.3(4.5) ≈ 60
- The output can be ANY number — 60, 60.5, even 105 if extrapolated too far.

PART 1 · IN PYTHON

Fitting Linear Regression in Python

```
from sklearn.linear_model import LinearRegression
import numpy as np

hours = np.array([1,2,3,4,5,6,7,8]).reshape(-1,1)
marks = np.array([35,40,50,55,65,70,78,85])

model = LinearRegression()
model.fit(hours, marks)

print(model.predict([[4.5]]))   # → about 60.1
print(model.score(hours, marks))  # R² → about 0.99
```

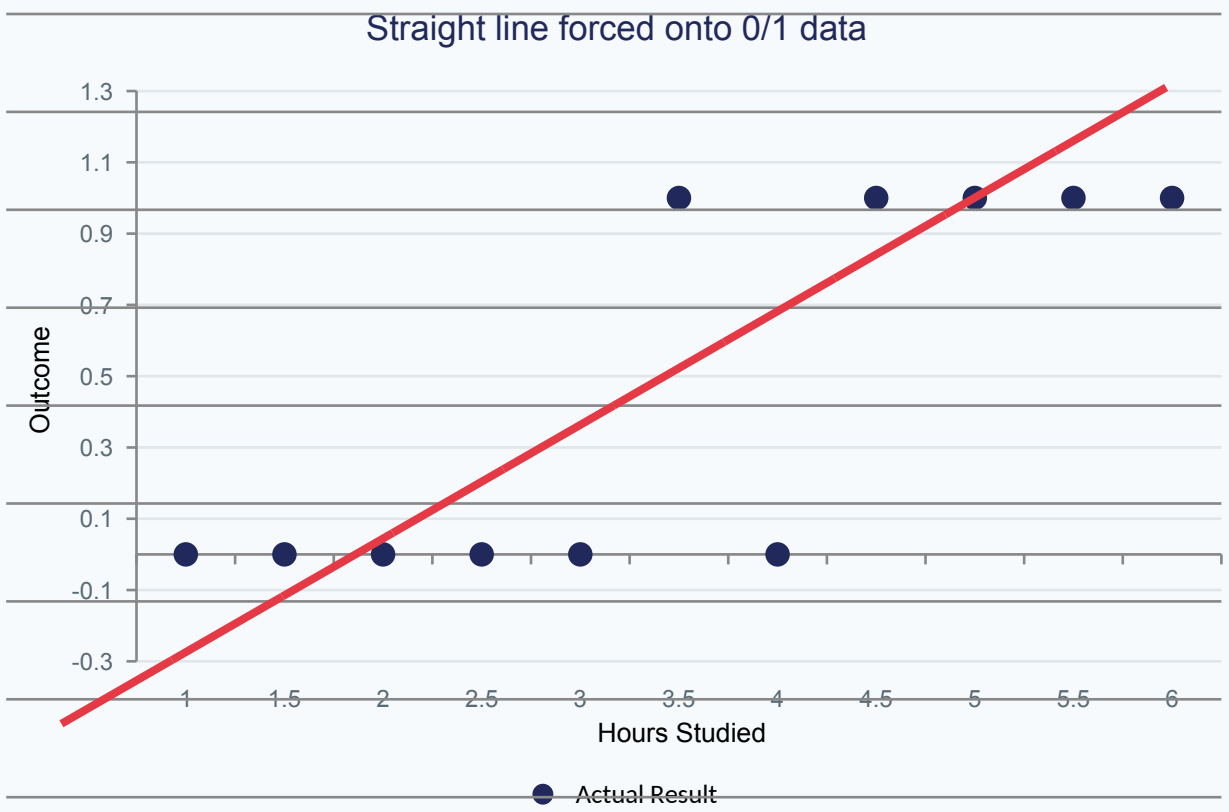
Output

A student who studies 4.5 hours is predicted to score ≈ 60 marks — a plain continuous number.

PART 2 · THE PROBLEM

Now Try to Predict Pass / Fail

Same idea — hours studied → outcome. But now the outcome is only 0 (Fail) or 1 (Pass), not a number.



⚠ THE PROBLEM

- A straight line has no floor or ceiling — it predicts -0.15 or 1.15.
- But a probability must stay between 0 and 1. Negative or >1 makes no sense.
- The line also treats a jump from 0.9→1.0 the same as 0.4→0.5 — real class boundaries aren't linear.
- Threshold at 0.5 gives poor, unstable class splits on this data.

PART 3 · THE FIX

Logistic Regression: Squeeze the Line Into [0, 1]

Predicts a PROBABILITY

Output is always between 0 and 1.

Fits an S-shaped curve

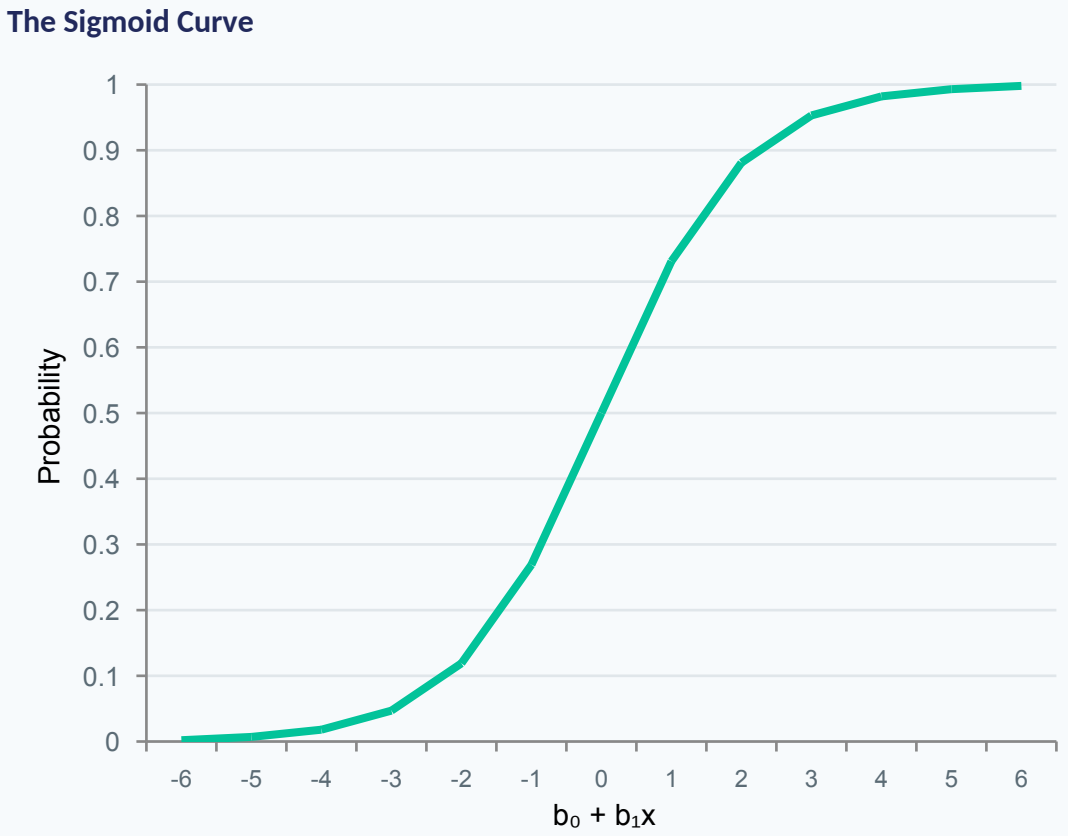
The sigmoid function bends the line into a bounded curve.

Answers "which class?"

e.g. Pass or Fail? Spam or Not spam? Default or not?

Equation (Sigmoid)

$$p = 1 / (1 + e^{-(b_0+b_1x)})$$



PART 3 · WORKED EXAMPLE

A Tiny Dataset: Study Hours → Pass/Fail

Hours Studied	Result (0=Fail, 1=Pass)
1	0
1.5	0
2	0
2.5	0
3	0
3.5	1
4	0
4.5	1
5	1
5.5	1
6	1

What the model learns

- Fits an S-curve so probability rises smoothly from 0 toward 1 as hours increase.
- Around 3–3.5 hours, predicted probability crosses 0.5 — the decision boundary.
- For a NEW student at 4 hours → model outputs p ≈ 0.68 (68% chance of passing).
- Apply a threshold (commonly 0.5): p ≥ 0.5 → predict “Pass”, else “Fail”.
- Notice: the raw output is always a valid probability — never negative, never above 1.

PART 3 · IN PYTHON

Fitting Logistic Regression in Python

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
import numpy as np

hours = np.array([1,1.5,2,2.5,3,3.5,4,4.5,5,5.5,6]).reshape(-1,1)
result = np.array([0,0,0,0,0,1,0,1,1,1,1])

model = LogisticRegression()
model.fit(hours, result)

print(model.predict_proba([[4]]))  # → [[0.32, 0.68]]
print(model.predict([[4]]))        # → [1]   (Pass)
print(accuracy_score(result, model.predict(hours)))  # → 0.91
```

Output

At 4 hours: 68% probability of passing → classified as “Pass”. Always a bounded probability, then a class label.

PART 3 · GOOD TO KNOW

Logistic Regression Has 3 Flavors

They all output probabilities via a sigmoid/softmax — they differ in how many classes and whether class order matters.

2 CLASSES

Binary Logistic Regression

The classic case — exactly two outcomes.

Example: Pass / Fail, Spam / Not spam

This is what we just built above.

3+ CLASSES · UNORDERED

Multinomial Logistic Regression

More than two outcomes with no natural ranking.

Example: Fruit is Apple / Banana / Mango

Uses softmax instead of a single sigmoid.

3+ CLASSES · ORDERED

Ordinal Logistic Regression

More than two outcomes WITH a meaningful order.

Example: Rating is Low / Medium / High

Respects the order when estimating probabilities.

Binary Logistic Regression

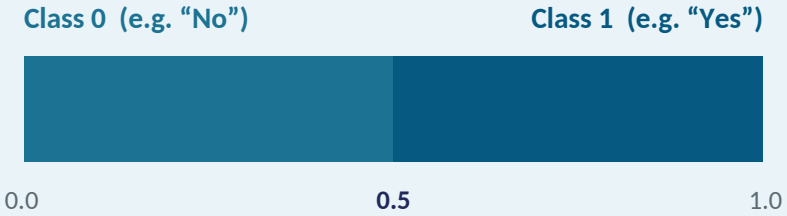
- The dependent variable has exactly two categories (e.g. 0 / 1, Yes / No).
- Models the probability of the “positive” class occurring, given the predictors.
- A threshold (commonly 0.5) converts the predicted probability into a class label.

$$P(Y=1 \mid X) = 1 / (1 + e^{-(\beta 0 + \beta 1 X)})$$

Classroom Examples

- Spam detection: spam / not spam
- Medical diagnosis: disease present / absent
- Customer churn: will leave / will stay
- Exam result: pass / fail

Decision Threshold



Predicted probability below 0.5 → classified as Class 0.
Predicted probability at or above 0.5 → classified as Class 1.

One equation is fit — the output splits observations into exactly two groups.

Multinomial Logistic Regression

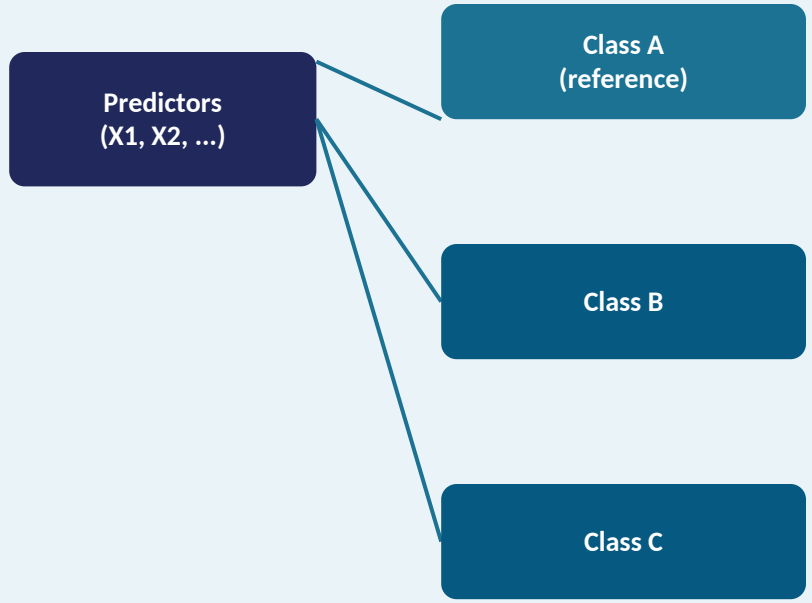
- The dependent variable has three or more categories with no natural order (nominal).
- Extends binary logistic regression by fitting K-1 log-odds equations against a chosen reference category.
- Each equation estimates the log-odds of one category relative to the reference.

$$ln[\ P(Y=k)\ /\ P(Y=ref)\ ]\ =\ \beta 0k + \beta 1k\ X1 + \ldots$$

Classroom Examples

- Mode of transport: car / bus / train
- Product category chosen by a shopper
- Political party preference in a survey
- Type of loan applied for: home / auto / personal

One Predictor Set → Multiple Classes



Each non-reference class gets its own equation, compared against the baseline (reference) class.

Binary vs. Multinomial: Quick Comparison

Aspect	Binary Logistic Regression	Multinomial Logistic Regression
Outcome categories	Exactly 2	3 or more (unordered)
Number of equations	1 logit equation	K − 1 logit equations
Reference category	Not required	One category set as baseline
Underlying function	Sigmoid function	Softmax (generalized sigmoid)
Typical example	Disease: positive / negative	Transport mode: car / bus / train
Coefficient interpretation	Odds ratio vs. the other class	Odds ratio vs. the reference class

DON'T SKIP THIS

How Do You Know the Model Is Good?

Fitting a model isn't the finish line — each type has its own way of measuring "good."

LINEAR REGRESSION

Measures how close predictions land to actual numbers

R^2 (R-squared)

% of variance the model explains. Closer to 1 is better.

MSE / RMSE

Average squared error between predicted and actual marks.

MAE

Average absolute error — easier to interpret in real units.

```
model.score(hours, marks)  →  R² ≈ 0.99
```

LOGISTIC REGRESSION

Measures how often predicted classes match actual classes

Confusion Matrix

Grid of correct vs incorrect Pass/Fail predictions.

Accuracy

% of all predictions that were correct.

Precision & Recall

How trustworthy "Pass" predictions are, and how many actual passes were caught.

```
accuracy_score(y_test, preds)  →  0.91
```

SIDE BY SIDE

Linear vs Logistic — Head to Head

Aspect	Linear Regression	Logistic Regression
Predicts	A continuous number	A probability (0 to 1)
Shape of fit	Straight line	S-shaped curve (sigmoid)
Typical question	“How much / how many?”	“Which class / how likely?”
Output range	$-\infty$ to $+\infty$	0 to 1
Error measure	Mean Squared Error	Log Loss (cross-entropy)
How it's fit	Least Squares (OLS)	Maximum Likelihood (MLE)
Evaluated with	R^2, MSE, MAE	Accuracy, Confusion Matrix, Precision/Recall
Example use	Predicting house price	Predicting spam vs not-spam

REAL-WORLD USE

When Should You Use Which?

LINEAR REGRESSION

Use when the answer is a number

- → House price from size, location
- → Salary from experience, education
- → Sales forecast from ad spend
- → Temperature or rainfall prediction
- → Exam score from study hours

LOGISTIC REGRESSION

Use when the answer is a category

- → Email: spam or not spam
- → Loan: default or not default
- → Tumor: malignant or benign
- → Customer: churn or stay
- → Student: pass or fail

Key Assumptions of Logistic Regression

1

Categorical Outcome

The dependent variable must be binary or categorical, not continuous.

2

Independence of Observations

Data points must be independent — no repeated measures or matched pairs.

3

Little / No Multicollinearity

Independent variables should not be highly correlated with each other.

4

Linearity of the Logit

Continuous predictors should be linearly related to the log-odds of the outcome.

5

Adequate Sample Size

Needs a reasonably large N — a common rule of thumb is ≥10 events per predictor.

6

No Strong Outliers

Extreme or highly influential data points can distort coefficient estimates.

Why That Formula? Odds & Log-Odds

The problem with modeling P directly

- Probability P is bounded between 0 and 1 — but a linear equation β0 + β1X is unbounded.
- Fitting P = β0 + β1X directly can predict impossible values below 0 or above 1.

Step 1 — Convert to Odds

$$Odds = P / (1 - P)$$

Removes the upper bound — odds range from 0 to ∞. But they're still stuck at a floor of 0, and skewed (not symmetric around “no effect”).

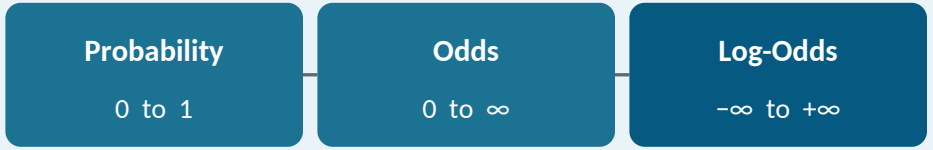
Step 2 — Take the Log

$$Log\text{-}Odds\ (Logit) = ln[\ P / (1 - P)\ ]$$

Removes the lower bound too — log-odds range freely from −∞ to +∞, just like β0 + β1X. Now the two sides of the equation finally match.

$$ln[\ P / (1\text{-}P)\ ] = \beta 0 + \beta 1 X1 + \ldots$$

Three Stages, Three Ranges



Only the log-odds scale is unbounded in both directions — the one thing a linear equation can match.

P	Odds	Log-Odds
0.10	0.11	−2.20
0.30	0.43	−0.85
0.50	1.00	0.00
0.70	2.33	0.85
0.90	9.00	2.20

Bonus: exp(β1) = the odds ratio for a 1-unit rise in X1 — directly interpretable.

PART 3 · UNDER THE HOOD

Why That Formula? Odds & Log-Odds

This is the part students often skip — but it's WHY logistic regression is still called “regression.”

Odds

Odds = p / (1 − p) — chance of passing vs. not passing.

Log-Odds (the “logit”)

Take the log of the odds: ln(p / (1−p)).

This part IS linear

$\ln(p / (1-p)) = b_0 + b_1 x$ — a straight line, just like linear regression!

Solve for p

Rearranging that line gives the sigmoid — that's where it comes from.

Reading a coefficient: if $b_1 = 0.9$, then e^0.9 ≈ 2.46 — each extra study hour multiplies the ODDS of passing by ~2.46×.

From Probability to Log-Odds

p	Odds p/(1−p)	Log-Odds
0.10	0.11	−2.20
0.30	0.43	−0.85
0.50	1.00	0.00
0.70	2.33	0.85
0.90	9.00	2.20

Notice: probability is squeezed into 0–1, but log-odds stretches out to −∞/+∞ — that unbounded scale is what a straight line can actually fit.

The Sigmoid (Logit) Function

Probability form

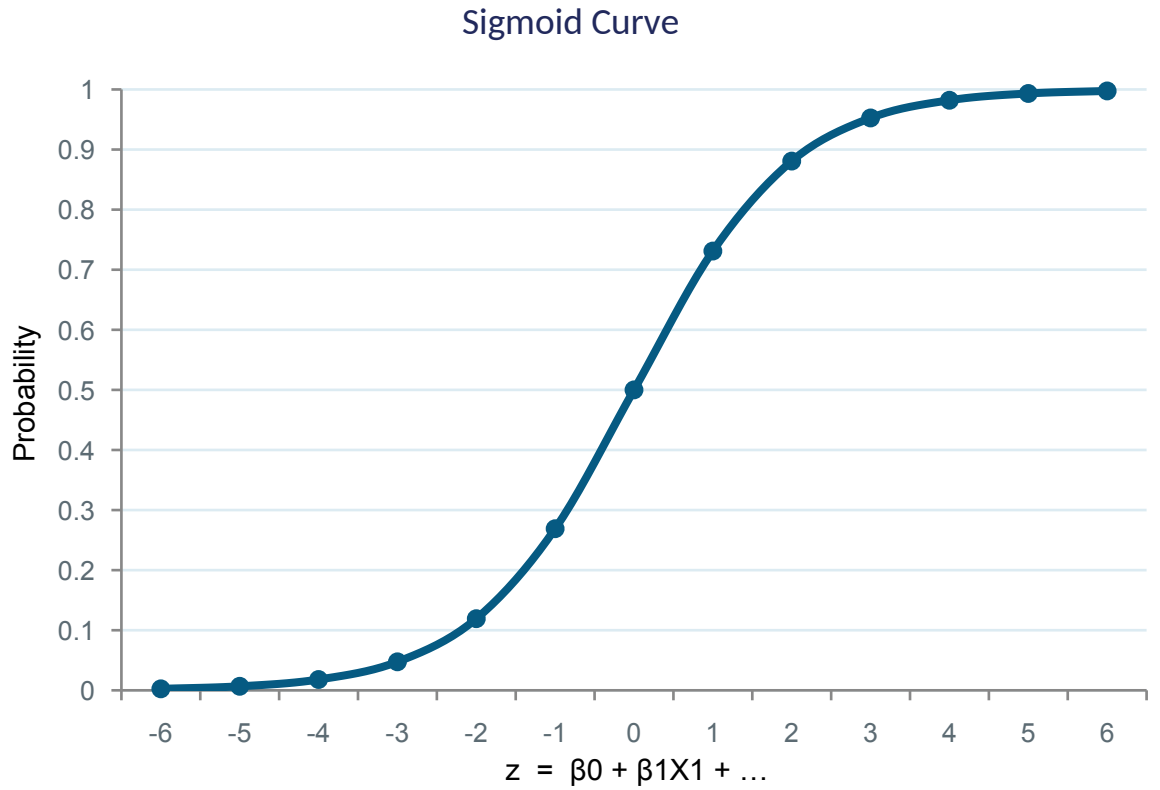
$$P(Y=1 \mid X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n)}}$$

The output is always squeezed between 0 and 1.

Logit (log-odds) form

$$\ln\left[\frac{P}{1-P}\right] = \beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n$$

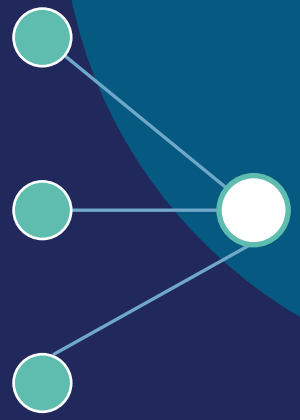
The logit is linear in the predictors — this is what logistic regression actually fits.



At z = 0, P = 0.5 → the standard classification threshold.

Perceptron & Artificial Neural Networks

From the biological neuron to Rosenblatt's Perceptron — how machines learned to mimic the mind



What We'll Cover

01 **Biological Neuron**

How nerve cells receive, process & transmit signals

02 **Artificial Neuron**

Synapses, summation junction & activation function

03 **Activation Functions**

Identity, step & threshold functions

04 **McCulloch–Pitts Model**

The earliest ANN implementation

05 **Rosenblatt's Perceptron**

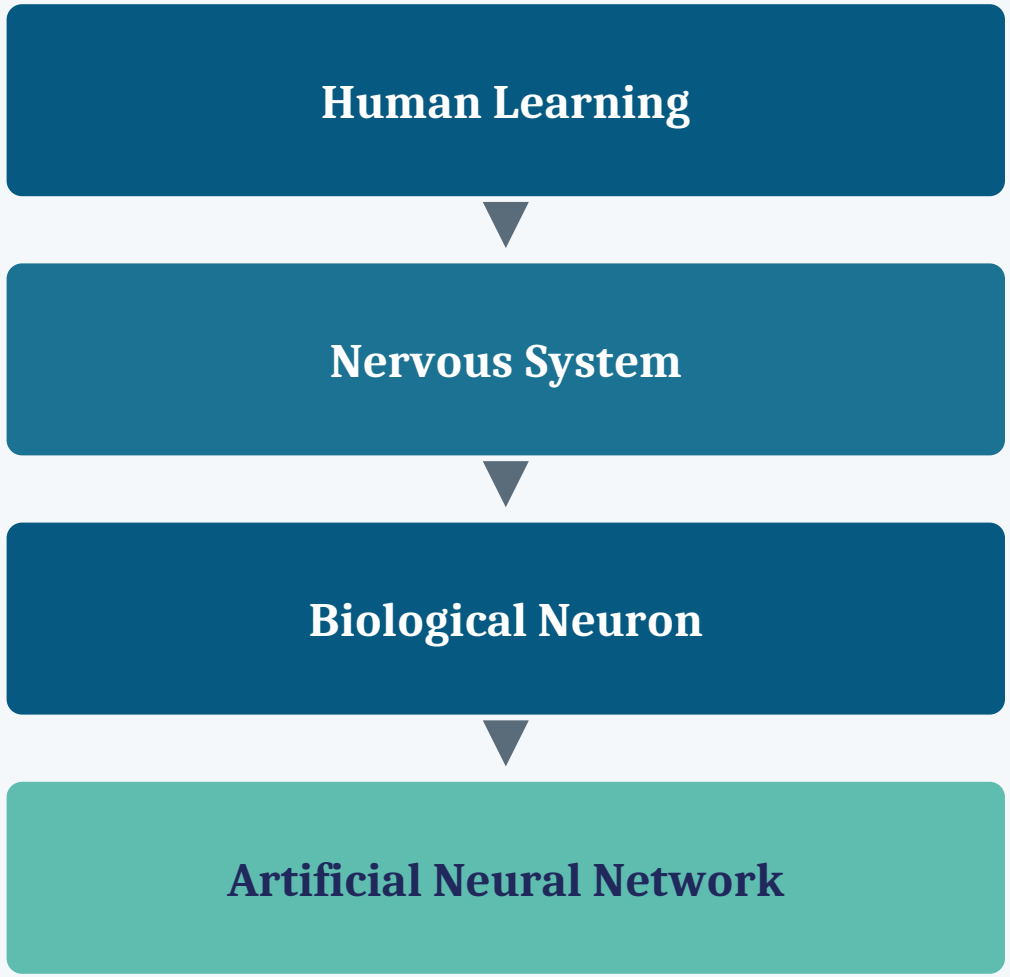
Math, classification & decision boundaries

06 **Linear Separability**

Where the perceptron succeeds — and fails

From Machine Learning to Neural Networks

Machine learning mimics the human learning process — a machine performs tasks and improves with experience, just as humans do. Having explored supervised, unsupervised and reinforcement learning,
the next step is to see how the human nervous system itself has been mimicked inside a computer.



The Human Nervous System

The nervous system coordinates every action of the human body by transmitting signals to and from its different parts. It has two main parts:

CNS

Central Nervous System (CNS)

Consists of the brain and spinal cord. It integrates all information, in the form of signals, from the different parts of the body.

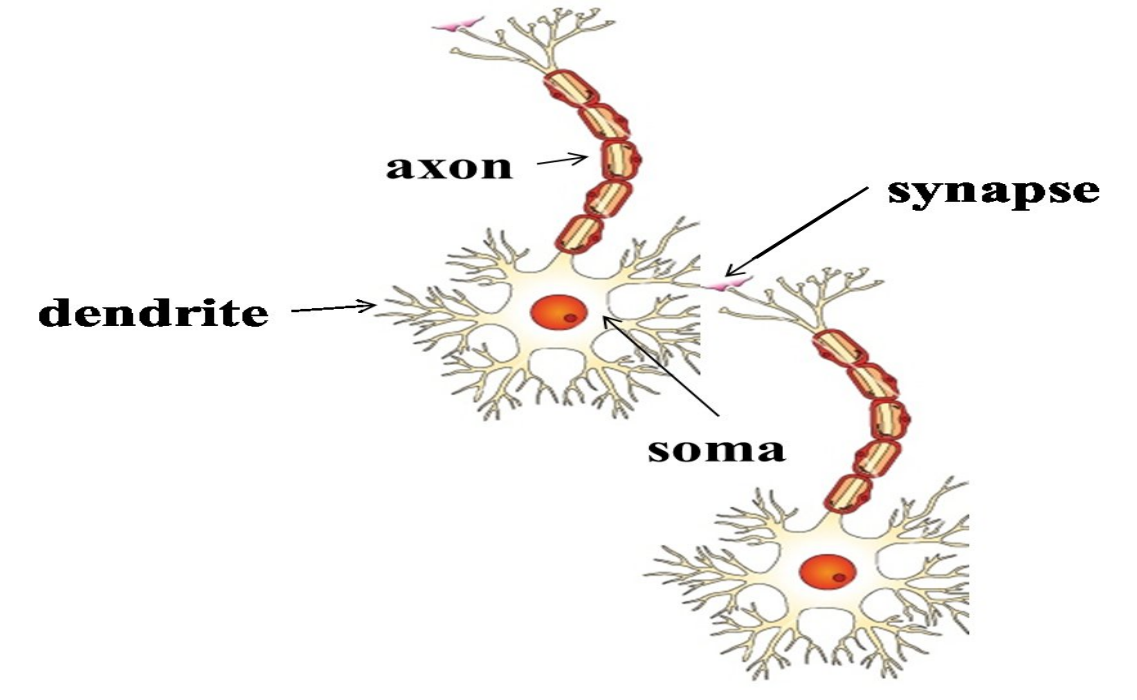
PNS

Peripheral Nervous System

Consists of nerves and ganglia outside the brain and spinal cord. It connects the CNS with the limbs and organs.

Structure of a Biological Neuron

A neuron receives, processes, and transmits chemical and electrical signals through three main parts:



1. **Dendrites**
   Receive signals from neighbouring neurons.

2. **Soma**
   Main body that accumulates incoming signals and 'fires' once enough signal has built up.

3. **Axon**
   Carries the fired signal through axon terminals to the dendrites of neighbouring neurons.

The small gap between one neuron's axon terminal and the next neuron's dendrite is the synapse — signals crossing it may be excitatory or inhibitory.

The Brain, By the Numbers

1.3 kg

Weight of the adult human brain

100 Billion

Neurons (10^{11}) in the human brain

10^{16}

Total synaptic connections

10^{5}

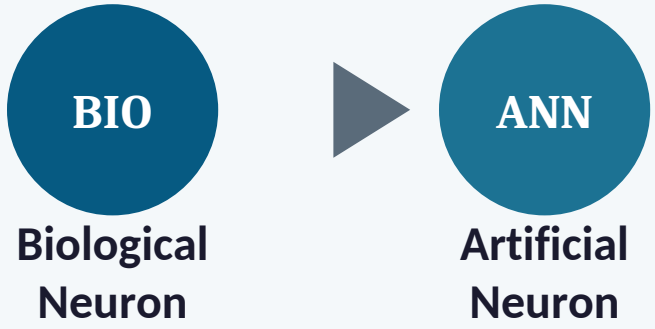
Other neurons each neuron connects to

Glial (glue) cells act as support cells for neurons — roughly 10× more numerous than neurons themselves, which make up only ~10% of all brain cells. Each axon is just 10–12 µm wide, and every synapse spans a gap only about a millionth of an inch.

The Birth of Artificial Neural Networks

The fascinating capability of the biological neural network — massive parallel processing built from billions of connected neurons — inspired the invention of the Artificial Neural Network (ANN).

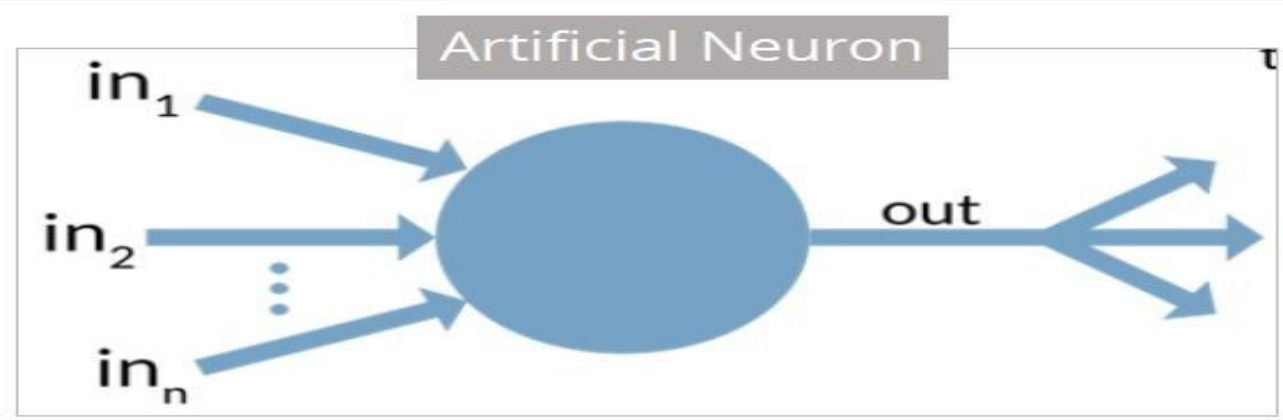
- An ANN is a machine designed to model the functioning of the nervous system — specifically, the neurons.
- The biological neuron is replicated in electronic or digital form.
- Digital / artificial neurons form the smallest processing units of an ANN.



"The only difference is that the biological form of neuron is replicated in the electronic or digital form of neuron."

Structure of an Artificial Neuron

Each artificial neuron has three major components:



1. **Synapses (weights)** — Each input xi connects through a synapse of weight wi. Positive weight = excitatory; negative weight = inhibitory.
2. **Summation Junction** — A linear combiner that adds the weighted inputs: ysum = Σ wi·xi
3. **Activation Function** — A 'squashing' function that produces an output only when the signal crosses a threshold: yout = f(ysum)

The Summation Junction

The summation junction is a linear combiner (adder) for all weighted input signals arriving at the neuron.

Without bias

$$y_{sum} = \sum_{i=1}^{n} w_i \cdot x_i$$

With bias 'b'

$$y_{sum} = b + \sum w_i \cdot x_i$$

Note: A neural network typically includes a bias which adjusts the input of the activation function. For simplicity, the base model ignores bias, treating it as an additional weighted input.

wi may be positive (excitatory effect) or negative (inhibitory effect) on the output of the summation junction, ysum.

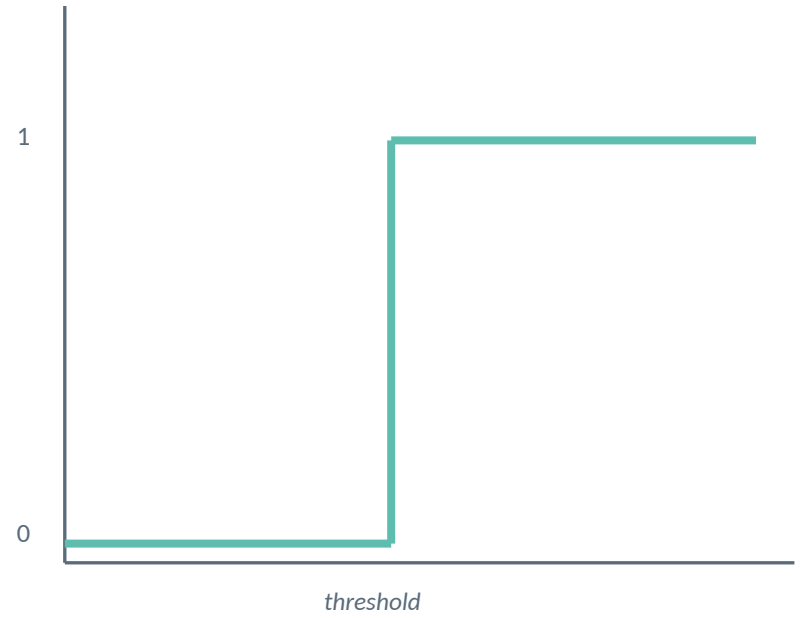
The Activation Function

Also called a 'squashing function', the activation function produces an output signal only when the input exceeds a specific threshold value — mirroring how a biological neuron only transmits a signal once its firing threshold is met.

$$yout = f(ysum)$$

The output of the activation function, yout, is the final signal the neuron passes on to the next layer or neuron.

Why 'Squashing'?



No signal below threshold — full signal once it's crossed.

Three Foundational Activation Functions

01

Identity Function

$$y_{out} = f(x) = x$$

Output equals the input exactly — used for the input layer.

02

Step Function

$$1 \text{ if } x \geq 0, \text{ else } 0$$

Fires (1) for zero or positive input; silent (0) otherwise.

03

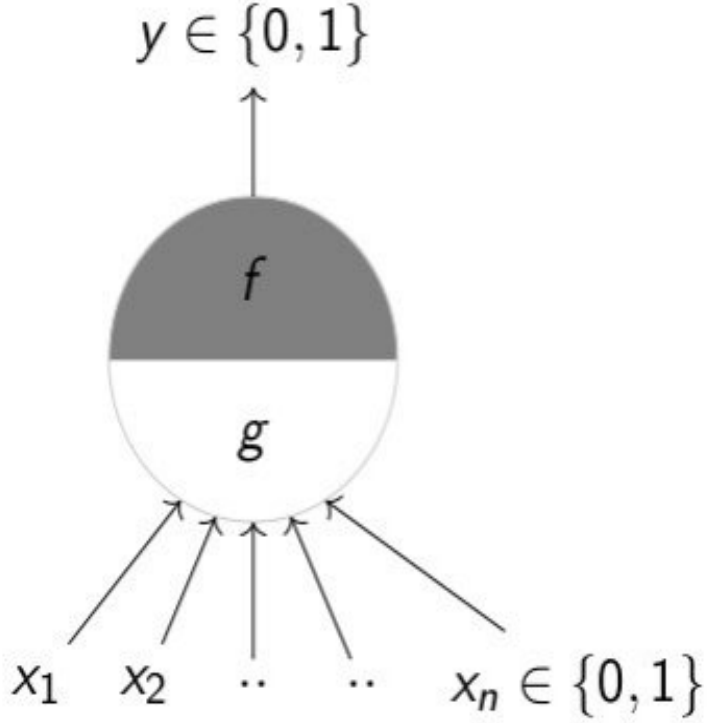
Threshold Function

$$1 \text{ if } x \geq \theta, \text{ else } 0$$

Like the step function, but fires only past a custom threshold θ.

McCulloch-Pitts Neuron Model and Thresholding logic

Proposed by: W. S. McCulloch (neuroscientist) and Walter Pitts (logician) in 1943.
Simplified Neuron Model: Created a basic representation of a neuron as a mathematical model.
Binary Output: The model outputs either **0** (no firing) or **1** (firing), based on input signals.
Input Threshold: Neuron fires when the sum of inputs exceeds a certain threshold.
Logic Function: Modeled neurons using logical operations (AND, OR, etc.).
Impact: The model laid the foundation for artificial neural networks and inspired computational neuroscience.

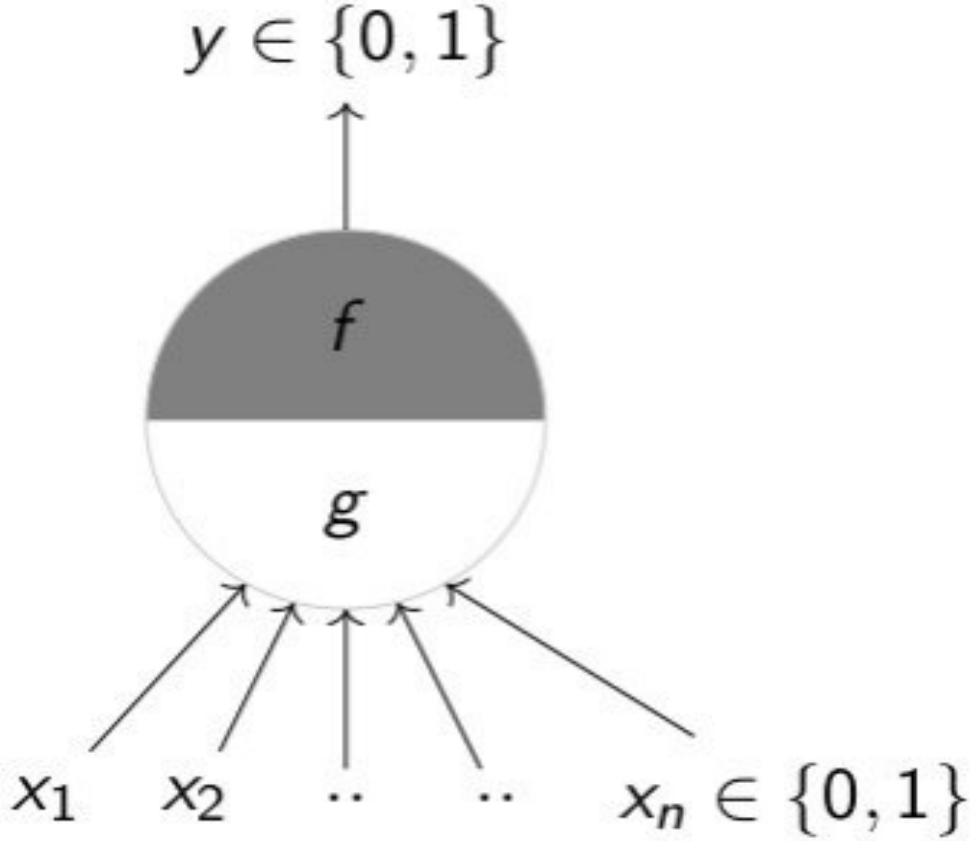


Working of the McCulloch-Pitts Neuron Model

- McCulloch (neuroscientist) and Pitts (logician) proposed a highly simplified computational model of the neuron (1943)
- **g** aggregates the inputs and the function **f** takes a decision based on this aggregation
- The inputs can be **excitatory** or **inhibitory**
- y = 0 if any xi is inhibitory, else excitatory
- θ is called the thresholding parameter
- This is called **Thresholding Logic**

$$g(x_1, x_2, ..., x_n) = g(\mathbf{x}) = \sum_{i=1}^{n} x_i$$

$$y = f(g(\mathbf{x})) = 1 \quad if \quad g(\mathbf{x}) \geq \theta$$
$$= 0 \quad if \quad g(\mathbf{x}) < \theta$$



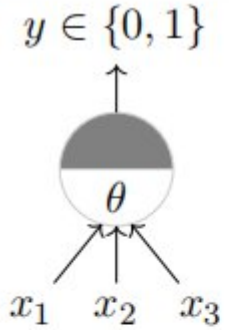
- **Input Signals:** The model receives multiple binary input signals (0 or 1).
- **Summing Function:** The model sums the inputs.
- **Threshold:** The sum is compared to a threshold value.

➢ If the sum is greater than or equal to the threshold, the neuron fires (output = 1).

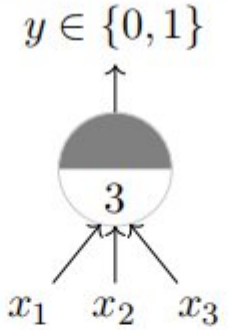
➢ If the sum is less than the threshold, the neuron does not fire (output = 0).

- **Output:** The final result is a binary output (0 or 1) based on the threshold comparison.
- **Excitatory inputs:** increase the sum, pushing the neuron closer to firing.It represented by positive weights (e.g., +1).
- **Inhibitory inputs:**decrease the sum, making it less likely for

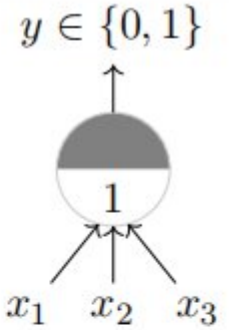
Implementing Boolean Functions with McCulloch-Pitts Neuron Model



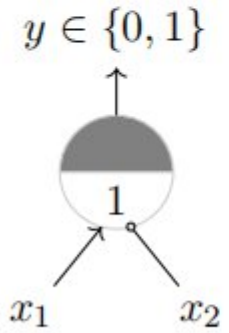
A McCulloch Pitts unit



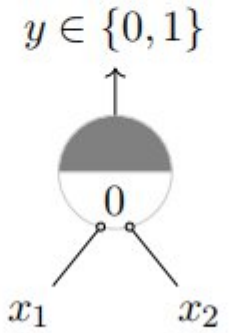
AND function



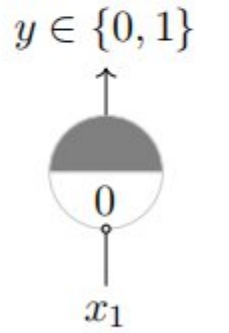
OR function



x_1 AND $!x_2$*



NOR function



NOT function

*circle at the end indicates inhibitory input: if any inhibitory input is 1 the output will be 0

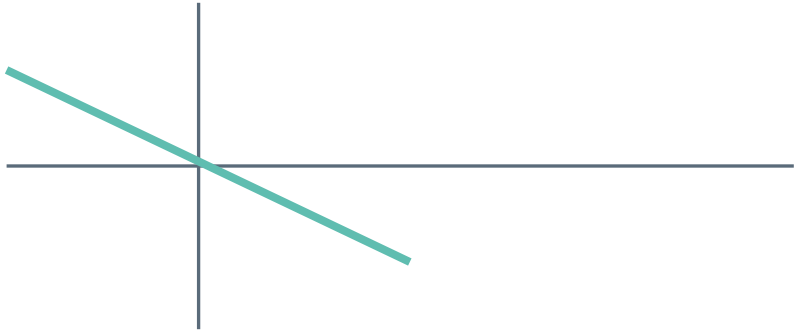
10.4.1 & 10.4.2

Identity Function & Step Function

Identity Function

yout = f(x) = x, for all x

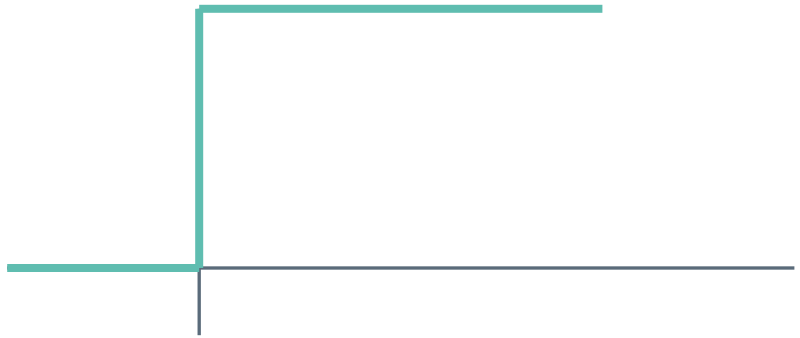
Used as the activation function for the input layer — the output remains exactly the same as the input, with no transformation applied.



Step Function

yout = 1 if x ≥ 0, 0 if x < 0

The most commonly used activation function. It gives 1 as output for zero or positive input, and 0 for negative input.

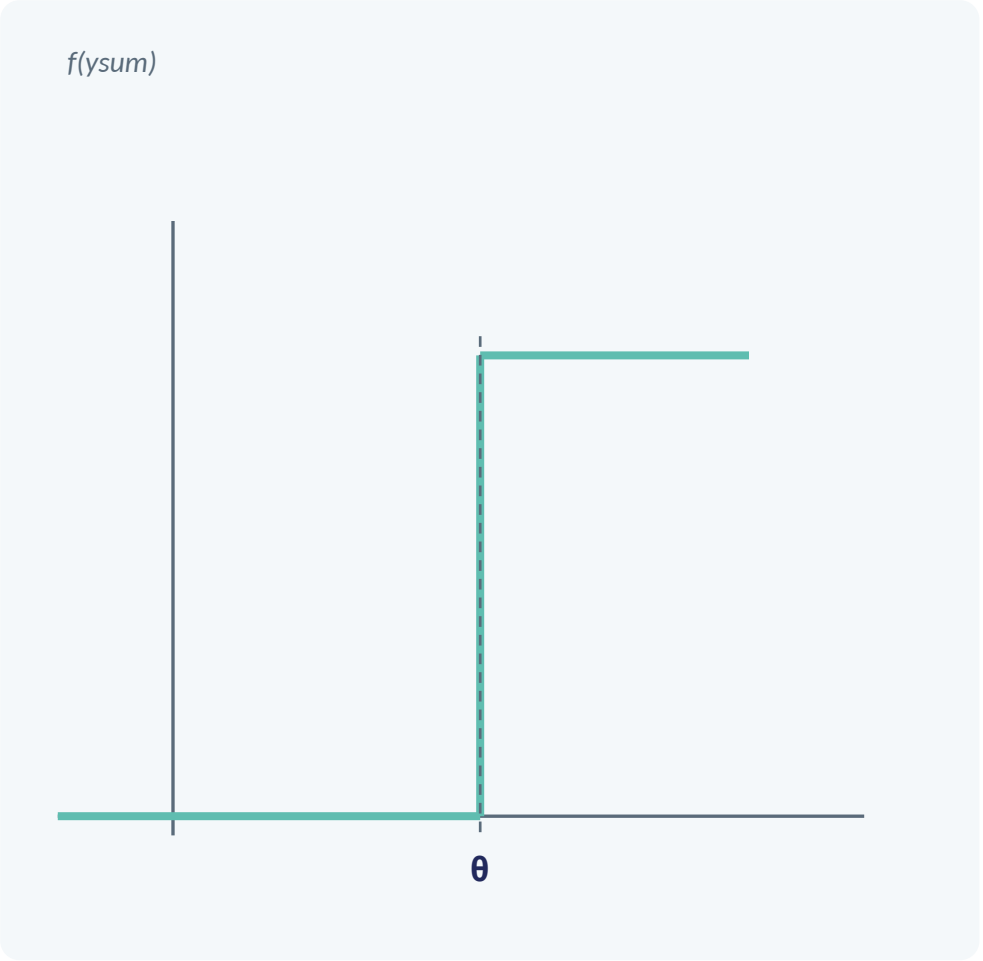


The Threshold Function

The threshold function behaves almost like the step function — the only difference is that a custom threshold value θ is used instead of 0.

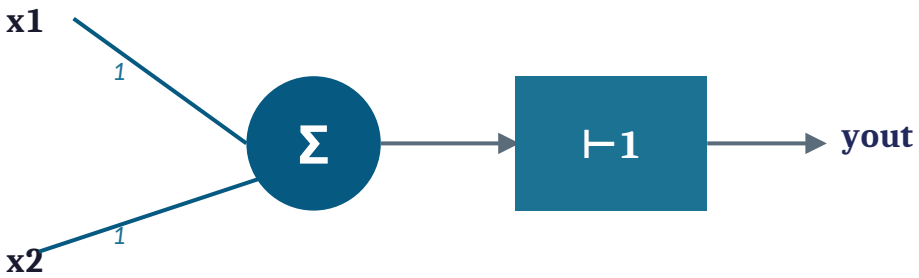
$$\text{yout} = f(\text{ysum}) = \begin{cases} 1, & x \geq \theta \\ 0, & x < \theta \end{cases}$$

θ (theta) is the threshold value that determines when the neuron 'fires' — this is the same concept the McCulloch–Pitts model builds on.



Does John Carry an Umbrella?

John carries an umbrella if it is sunny OR if it is raining. Using x1 = 'Is it raining?' and x2 = 'Is it sunny?', both weights are set to 1 and the threshold to 1.



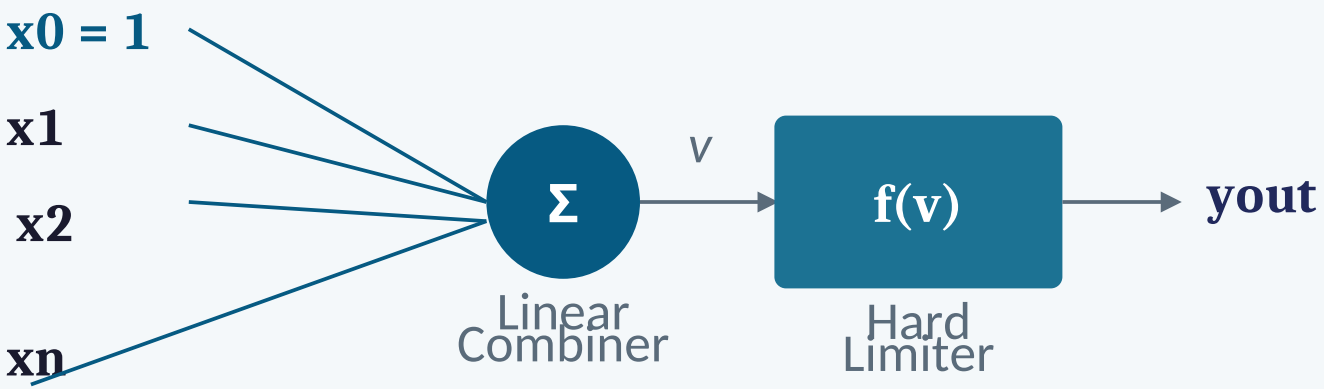
yout = 1 → John carries the umbrella

Situation	x1	x2	ysum	yout
1 — no rain, no sun	0	0	0	**0**
2 — sunny only	0	1	1	**1**
3 — raining only	1	0	1	**1**
4 — both	1	1	2	**1**

The result is the logical OR function — implemented purely through weights and a threshold.

Rosenblatt's Perceptron

Built around the McCulloch–Pitts neural model, Rosenblatt's perceptron receives a set of inputs x1, x2, ..., xn, computes their weighted sum, and passes it through a hard limiter.



Goal
Classify inputs into two classes, c1 and c2, using a simple decision rule based on the sign of yout.

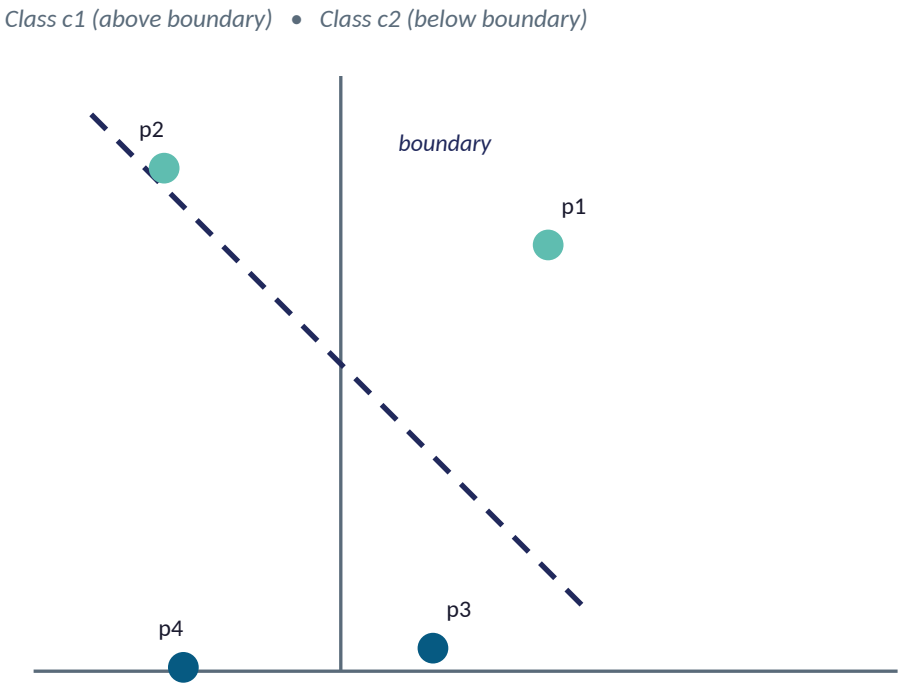
yout = +1 ⟶ c1
yout = −1 ⟶ c2

An adjustable bias w0 is attached to a dummy input x0 (always = 1), letting the decision boundary shift instead of always passing through the origin.

Classifying Points With the Perceptron

Using boundary $2x1 + x2 = 8$ (w0=−2, w1=½, w2=¼): p1(5,2) and p2(−1,12) belong to c1; p3(3,−5) and p4(−2,−1) belong to c2.

Point	v = Σ wixi	yout	Class
p1 = (5, 2)	−2+½(5)+¼(2) = 1	1	c1
p2 = (−1, 12)	−2+½(−1)+¼(12) = 0.5	1	c1
p3 = (3, −5)	−2+½(3)+¼(−5) = −1.75	0	c2
p4 = (−2, −1)	−2+½(−2)+¼(−1) = −3.25	0	c2

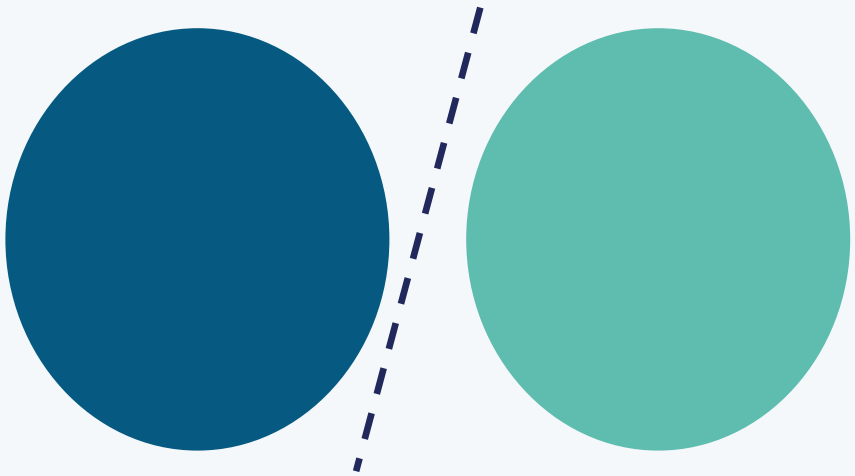


Feature	McCulloch-Pitts Neuron	Perceptron
Purpose	Theoretical neuron model	Binary classification
Inputs	Binary (0 or 1)	Real-valued
Weights	Fixed, predefined	Learnable via training
Bias	Not included	Included
Learning Mechanism	None	Yes (supervised learning)
Output	Binary (0 or 1)	Binary (0 or 1)

Linear Separability: The Perceptron's Boundary

For synaptic weights to be found appropriately, one key assumption must hold: the training data's classes must be linearly separable.

✓ Linearly Separable



A single straight line (or hyperplane) can fully separate class c1 from class c2 — the perceptron can solve this.

✗ Non-Linearly Separable



Interlocking / curved regions cannot be split by any single straight line — the classification problem cannot be solved by a perceptron alone.

Perceptron Learning Algorithm

Algorithm Perceptron learning algorithm

Input:

A set of training examples $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$

Learning rate $0 < \alpha < 1$

Number of epochs $epochs$

Initialize the weight vector $\mathbf{w}$ with random values
Initialize the bias $b \leftarrow 0$
for $i \leftarrow 1$ to $epochs$ **do**
    $err \leftarrow 0$ ▷ The number of misclassifications
    for each training example $(\mathbf{x}_i, y_i) \in D$ **do**
        $z_i \leftarrow \mathbf{w}^T \mathbf{x}_i + b$
        $o_i \leftarrow \begin{cases} 1 & z_i \geq 0 \\ 0 & \text{otherwise} \end{cases}$
        if $y_i \neq o_i$ **then**
            $\mathbf{w} \leftarrow \mathbf{w} + \alpha(y_i - o_i)\mathbf{x}_i$
            $b \leftarrow b + \alpha(y_i - o_i)$
            $err \leftarrow err + 1$
    if $err = 0$ **then break**

Perceptron Learning Algorithm

1. **Initialization**:
   - Initialize the weight vector w and the bias b (often set to small random values or zero).
   - Define a learning rate η (a small positive number).
2. **Forward Pass**:
   - Compute the output for an input vector x:

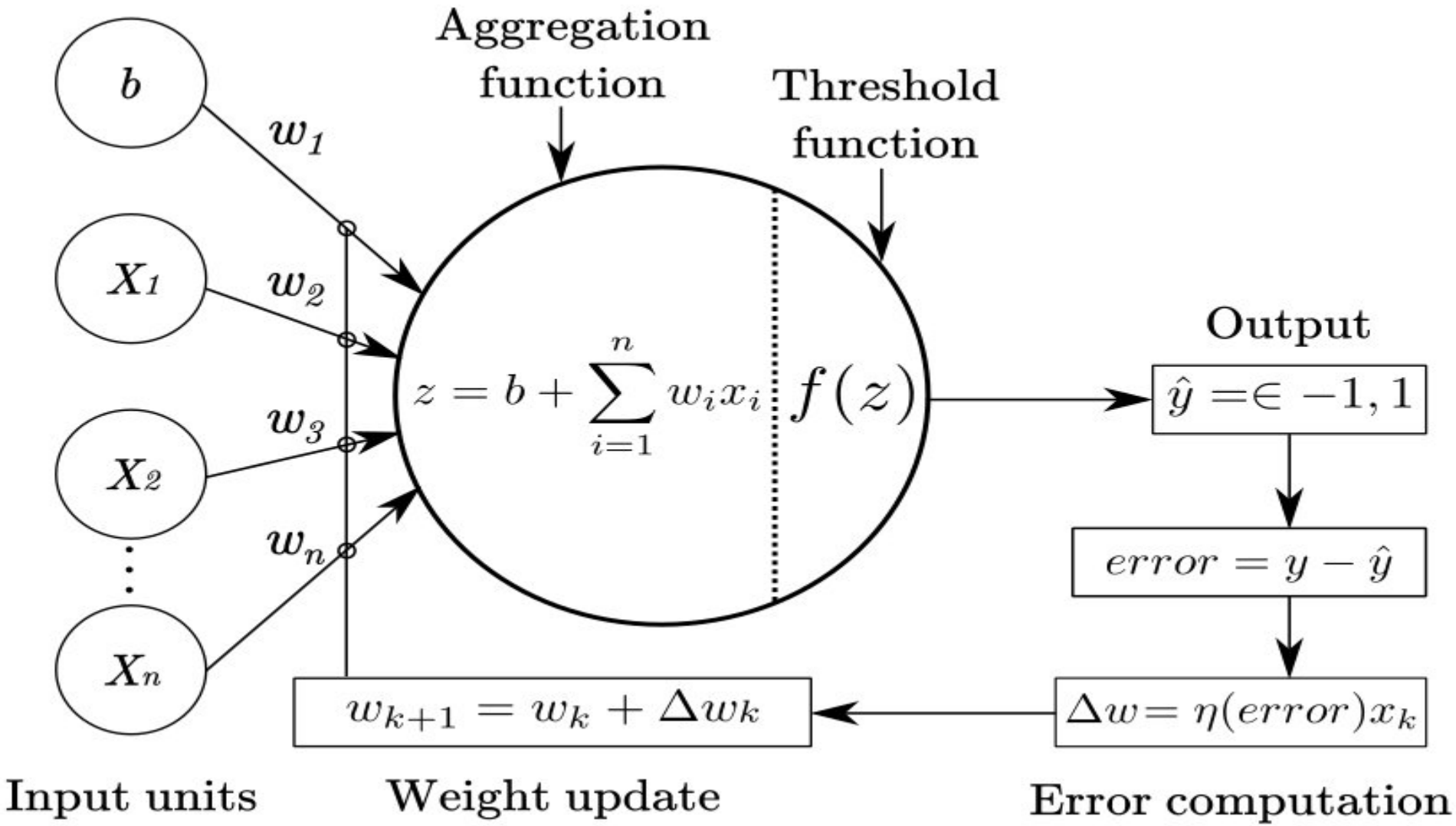
     $$y = \text{sign}(w^T x + b)$$

     where $\text{sign}(\cdot)$ is a step activation function.
3. **Error Calculation**:
   - Compare the predicted output y with the actual target t:

     $$\text{Error} = t - y$$
4. **Weight and Bias Update**:
   - If the prediction is incorrect ($y \neq t$), update the weights and bias:

     $$w \leftarrow w + \eta \cdot t \cdot x$$

     $$b \leftarrow b + \eta \cdot t$$
5. **Repeat**:
   - Iterate through all the training samples until the perceptron correctly classifies all data points or a maximum number of iterations is reached.


b
X1
X2
Xn
w1
w2
w3
wn
Aggregation function
Threshold function
$z = b + \sum_{i=1}^{n} w_i x_i$
$f(z)$
Output
$\hat{y} = \in -1, 1$
$error = y - \hat{y}$
$\Delta w = \eta(error)x_k$
$w_{k+1} = w_k + \Delta w_k$
Input units
Weight update
Error computation

Strengths of Perceptron

- **Simple Architecture**: Easy to understand and implement.
- **Fast Training:** Learns quickly with lightweight computations.
- **Linearly Separable Data:** Works effectively when classes can be separated by a straight line.
- **Low Resource Requirement:** Requires less memory and computational power.
- **Suitable for Small Datasets:** Can be efficiently used with simple and small datasets.
- **Foundation for Deep Learning:** Forms the basic concept behind multilayer neural networks.

Limitations of Perceptron

- **No Non-Linear Learning:** Cannot solve complex problems like XOR.
- **Binary Output:** Mainly designed for two-class classification.
- **No Hidden Layers:** Cannot learn complex or abstract patterns.
- **Sensitive to Feature Scaling:** Works better when input features are properly normalized.
- **Limited Real-World Applications:** Too simple for complex tasks such as image recognition and NLP.

Applications

- **Binary Classification:** Used for simple Yes/No or two-class problems.
- **Logic Gate Modeling:** Can implement gates such as AND and NAND.
- **Simple Pattern Recognition:** Identifies patterns that are linearly separable.
- **Feature Importance:** Weights show how strongly each feature influences the output.
- **Educational Purpose:** Helps in understanding the basics of neural networks and machine learning.